

```
// === BÀI 1: INTERFACE MAIL ===
```

```
interface Mail {  
    void sendMail();  
    void log();  
}
```

```
class MailImpl implements Mail {  
    @Override  
    public void sendMail() { System.out.println("Email da duoc gui thanh cong!"); }  
    @Override  
    public void log() { System.out.println("Log: Ghi lai lich su gui mail."); }  
}
```

```
// === BÀI 2: INTERFACE SHAPE ===
```

```
interface Shape {  
    void draw();  
}
```

```
class Htron implements Shape {  
    String color = "red";  
    public void draw() { System.out.println("Vẽ Hình Tron mau " + color); }  
}
```

```
class HCN implements Shape {  
    String color = "blue";  
    public void draw() { System.out.println("Vẽ Hình Chu Nhat mau " + color); }  
}
```

```
// === BÀI 3: QUẢN LÝ NHÂN VIÊN (IEmployee) ===
```

```
interface IEmployee {  
    int calculateSalary();  
    String getName();  
}
```

```
abstract class Employee implements IEmployee {  
    protected String name;  
    protected int paymentPerHour;  
  
    public Employee(String name, int paymentPerHour) {  
        this.name = name;  
        this.paymentPerHour = paymentPerHour;  
    }
```

```
    @Override  
    public String getName() { return name; }  
}
```

```
class PartTimeEmployee extends Employee {  
    private int workingHours;  
    public PartTimeEmployee(String n, int p, int h) { super(n, p); this.workingHours = h; }  
    @Override  
    public int calculateSalary() { return workingHours * paymentPerHour; }  
}
```

```
class FullTimeEmployee extends Employee {  
    public FullTimeEmployee(String n, int p) { super(n, p); }  
    @Override  
    public int calculateSalary() { return 8 * paymentPerHour; }
```

```
}
```

```
// === BÀI 4: HỆ THỐNG DI CHUYỂN (Moveable) ===
```

```
interface Moveable {  
    void moveUp(); void moveDown(); void moveLeft(); void moveRight();  
}
```

```
class MoveablePoint implements Moveable {  
    int x, y, xSpeed, ySpeed;  
    public MoveablePoint(int x, int y, int xs, int ys) {  
        this.x = x; this.y = y; this.xSpeed = xs; this.ySpeed = ys;  
    }  
    @Override public String toString() { return "(" + x + "," + y + ")"; }  
    @Override public void moveUp() { y -= ySpeed; }  
    @Override public void moveDown() { y += ySpeed; }  
    @Override public void moveLeft() { x -= xSpeed; }  
    @Override public void moveRight() { x += xSpeed; }  
}
```

```
class MoveableCircle implements Moveable {  
    private int radius;  
    private MoveablePoint center;  
    public MoveableCircle(int x, int y, int xs, int ys, int r) {  
        this.center = new MoveablePoint(x, y, xs, ys);  
        this.radius = r;  
    }  
    @Override public String toString() { return "Circle at " + center.toString() + " with R=" +  
radius; }  
    @Override public void moveUp() { center.moveUp(); }
```

```

@Override public void moveDown() { center.moveDown(); }

@Override public void moveLeft() { center.moveLeft(); }

@Override public void moveRight() { center.moveRight(); }
}

```

// === BÀI 5: HÌNH HỌC & RESIZABLE ===

```

interface GeometricObject {

    double getPerimeter();

    double getArea();

}

```

```

interface Resizable {

    void resize(int percent);

}

```

```

class Circle implements GeometricObject {

    protected double radius;

    public Circle(double r) { this.radius = r; }

    @Override public double getPerimeter() { return 2 * Math.PI * radius; }

    @Override public double getArea() { return Math.PI * radius * radius; }

    @Override public String toString() { return "Circle[r=" + radius + "];" }

}

```

```

class ResizableCircle extends Circle implements Resizable {

    public ResizableCircle(double r) { super(r); }

    @Override

    public void resize(int percent) { radius *= (percent / 100.0); }

    @Override

    public String toString() { return "ResizableCircle[" + super.toString() + "];" }

}

```

```
}
```

```
// === HÀM MAIN CHẠY TẤT CẢ ===
```

```
public class B9 {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("--- Bai 1: Mail ---");
```

```
        MailImpl m = new MailImpl(); m.sendMail();
```

```
        System.out.println("\n--- Bai 2: Shape ---");
```

```
        new HCN().draw();
```

```
        System.out.println("\n--- Bai 3: Employee ---");
```

```
        IEmployee e = new PartTimeEmployee("Tam", 50000, 5);
```

```
        System.out.println(e.getName() + " Salary: " + e.calculateSalary());
```

```
        System.out.println("\n--- Bai 4: Moveable ---");
```

```
        MoveableCircle mc = new MoveableCircle(10, 10, 2, 2, 5);
```

```
        mc.moveRight();
```

```
        System.out.println(mc.toString());
```

```
        System.out.println("\n--- Bai 5: Resizable ---");
```

```
        ResizableCircle rc = new ResizableCircle(10.0);
```

```
        rc.resize(50); // Giảm bán kính xuống 50%
```

```
        System.out.println(rc.toString() + " Area: " + rc.getArea());
```

```
    }
```

```
}
```